
Physics Listの使用方法

演習パッケージ : P08_PhysicsLists

Geant4 11.3準拠

Geant4講習会資料: HEP/NP/Space 2025

本資料に関する注意

- 本資料の知的所有権は、高エネルギー加速器研究機構およびGeant4 collaborationが有します
- 以下のすべての条件を満たす場合に限り無料で利用することを許諾します
 - 学校、大学、公的研究機関等における教育および非軍事目的の研究開発のための利用であること
 - ・ Geant4の開発者はいかなる軍事関連目的へのGeant4の利用を拒否します
 - このページを含むすべてのページをオリジナルのまま利用すること
 - ・ 一部を抜き出して配布したり利用してはいけません
 - 誤字や間違いと疑われる点があれば報告する義務を負うこと
- 商業的な目的での利用、出版、電子ファイルの公開は許可なく行えません

演習トピックス

[概要] P08_PhysicsListsプログラムを使い、物理過程のプロセスの設定方法を学ぶ

[目次]

1. P08_PhysicsLists/Shielding

- Radioactive Decayの扱い

2. P08_PhysicsLists/Factory

- Physics Listsのカスタマイズ: G4PhysListFactoryを使う

3. P08_PhysicsLists/MyPhysList

- 独自のPhysics Listsの作成

P08_PhysicsLists/Shielding

Radioactive Decayの扱い

演習トピックス

[概要] [P08_PhysicsLists](#)プログラムを使い、物理過程のプロセスの設定方法を学ぶ

[目次]



1. [P08_PhysicsLists/Shielding](#)
 - Radioactive Decayの扱い
2. [P08_PhysicsLists/Factory](#)
 - Physics Listsのカスタマイズ: G4PhysListFactoryを使う
3. [P08_PhysicsLists/MyPhysList](#)
 - 独自のPhysics Listsの作成

P08_PhysicsLists/Shieldingプログラムの概要

■ 演習プログラムの目的

- 今までの演習で使用したPhysics ListはFTFP_BERTでHEP/Spaceのシミュレーションに最適
- しかし、radioactive decayを考慮したい場合、Shieldingなど放射性崩壊が組込まれているPhysics Listを使う必要がある
- 本演習ではFTFP_BERTをShieldingに交換する手法を学び、Reference Physics Listの切り替えが簡単であることを理解する

■ プログラムの構成

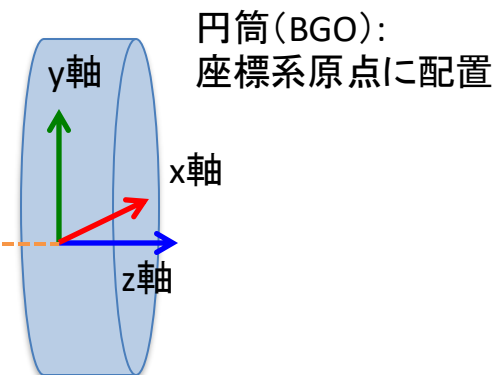
- mainプログラム: P06_Geometry/BGO_Oneと同一
- ジオメトリ: P06_Geometry/BGO_Oneと同一
- PhysicsList: Shielding (FTFP_BERTから変更)
- PrimaryGenerator: P06_Geometry/BGO_Oneと同一 (Particle Gun)

■ 組み込まれているジオメトリと粒子発生機能

- 初期粒子はマクロ(primaryGeneratorSetup.mac)に設定済み

Radioactive Source ^{137}Cs

設定位置: $x = 0, y = 0, z = -5 \text{ cm}$



P08_PhysicsLists/Shielding プログラムのビルド

課題:1 プログラム構成を理解する

- P06_Geometry/BGO_Oneがそのままコピーされている
- ただし、Application_Main.ccでFTFP_BERTの代わりにShieldingがPhysicsListとして設定されている

課題:2 P08_PhysicsLists/Shieldingのソースコードをビルドして実行する

1) P08_PhysicsLists/Sheildingのディレクトリに移行する

```
$ cd ~/Geant4Tutorial_2025/P08_PhysicsLists/Shielding
$ ls
source/  util/  TestBench/
```

← TestBenchのPrimaryGeneratorSetup.macには¹³⁷Csが設定されている

2) ソースをビルド

```
$ ./util/Build.sh
$ ls
bin/  build/  source/  util/  TestBench/
```

← Scriptを走らせるとアプリ実行環境が完成

← ソースのビルドでbin、buildが作られた

3) TestBench(作業ディレクトリ) に移行、プログラムを実行

```
$ cd TestBench
$ ../bin/Application_Main
```

P08_PhysicsLists/Shieldingプログラムを使う

課題:1 実行されたプログラムを使う

1) Qt画面のコマンド窓に以下を入力しアプリ実行

```
/control/execute GlobalSetup.mac
```

または、Qtのファイルアイコンをクリック

2) Qtコマンド窓に以下のコマンドを入力

```
/run/beamOn 1
```

- 右図の様なCs¹³⁷の崩壊過程が見える

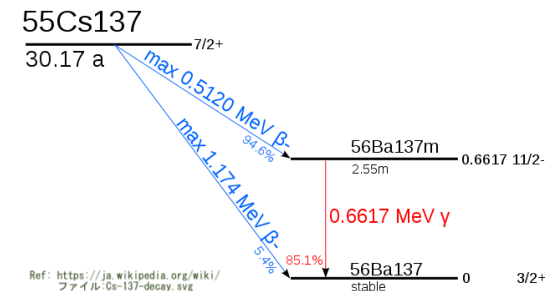
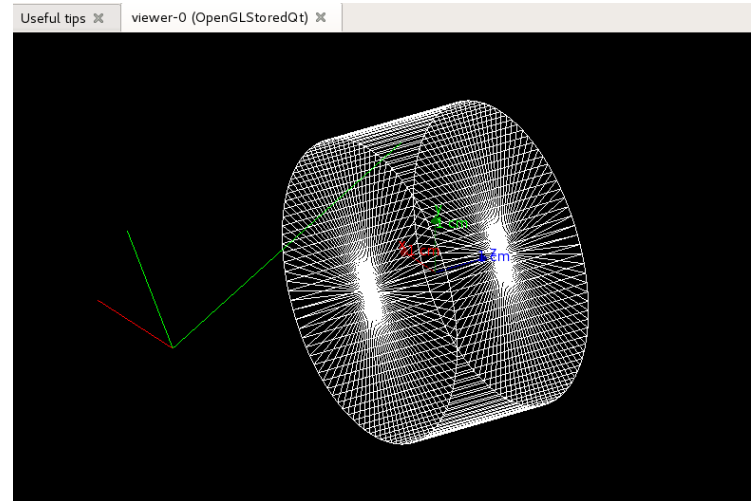
[注] TestBench下のマクロのprimaryGeneratorSetup.macにCs¹³⁷が初期粒子として以下のように設定されている

```
## Particle type
/gun/particle ion
/gun/ion 55 137

## Enable radioactive decay of long-decay-time elements
/process/had/rdm/thresholdForVeryLongDecayTime 1.0e+60 year
```

このコマンドの必要性

- Geant4 11.2から、寿命が1年以上の放射性イオンは崩壊させないのがdefault設定となった
- 長寿命のイオンを崩壊させるには、このコマンドで寿命制限を変更する必要がある
- 上記の1.0e+60 yearは、寿命がこの値以上を持つイオンは崩壊させないことを意味する(すなわち、全ての放射性イオンを崩壊させる)



[Ref]
Book for Application Developers
- Tracking and Physics
- Physics Processes

3) アプリを終了 `exit`

次の課題では、実行したプログラムのApplication_Main.ccのコードを読み、実装手法を学ぶ

P08_PhysicsLists/Shielding : Application_Main.ccの内容

課題:1 BGO_OneのApplication_Main.ccとShieldingのApplication_Main.ccをcolordiffコマンドで比較

```
$ cd ../source/  
$ colordiff -y --width=160 ../../P06_Geometry/BGO_One/Application_Main.cc Application_Main.cc
```

[注] colordiffのversionで出力形式が異なる可能性がある

課題:2 colordiffの結果から変更された箇所を確認する

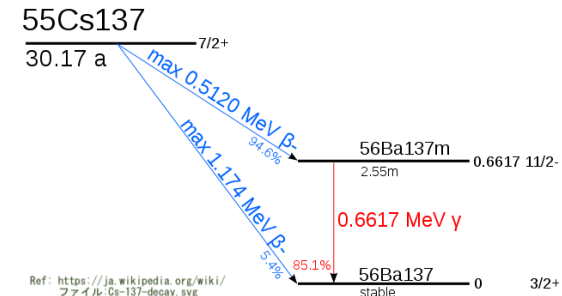
変更前ファイル	変更後ファイル
<pre>//+++++ // Geant4 Tutorial for Hep/Space/Medicine Users //+++++ #include "Geometry.hh" #include "UserActionInitialization.hh" #include "G4RunManager.hh" #include "G4UIManager.hh" #include "G4VisExecutive.hh" #include "G4UIExecutive.hh" #include "FTFP_BERT.hh" //----- int main(int argc, char** argv) //----- { // Construct a run manager auto runManager = new G4RunManager{}; // Construct a run manager whose type is specified by the environment variable auto runManager = G4RunManagerFactory::CreateRunManager(); // Set up mandatory user initialization: Physics-List runManager->SetUserInitialization(new FTFP_BERT{}); // Set up user initialization: User Actions runManager->SetUserInitialization(new UserActionInitialization{}); // Initialize G4 kernel runManager->Initialize(); // Create visualization environment auto visManager = new G4VisExecutive{}; visManager->Initialize(); // Start interactive session auto uiExec = new G4UIExecutive{ argc, argv }; G4UIManager* uiManager = G4UIManager::GetUIpointer(); uiManager->ApplyCommand("/control/execute GlobalSetup.mac"); uiExec->SessionStart(); // Job termination delete uiExec; delete visManager; delete runManager; return 0; }</pre>	<pre>//+++++ // // Geant4 Tutorial for Hep/Space/Medicine Users //+++++ #include "Geometry.hh" #include "UserActionInitialization.hh" #include "G4RunManager.hh" #include "G4UIManager.hh" #include "G4VisExecutive.hh" #include "G4UIExecutive.hh" #include "Shielding.hh" //----- int main(int argc, char** argv) //----- { // Construct a run manager auto runManager = new G4RunManager{}; // Construct a run manager whose type is specified by the environment variable auto runManager = G4RunManagerFactory::CreateRunManager(); // Set up mandatory user initialization: Physics-List runManager->SetUserInitialization(new Shielding{}); // Set up user initialization: User Actions runManager->SetUserInitialization(new UserActionInitialization{}); // Initialize G4 kernel runManager->Initialize(); // Create visualization environment auto visManager = new G4VisExecutive{}; visManager->Initialize(); // Start interactive session auto uiExec = new G4UIExecutive{ argc, argv }; G4UIManager* uiManager = G4UIManager::GetUIpointer(); uiManager->ApplyCommand("/control/execute GlobalSetup.mac"); uiExec->SessionStart(); // Job termination delete uiExec; delete visManager; delete runManager; return 0; }</pre>

P08_PhysicsLists/Shieldingプログラム: Cs137崩壊過程のstep情報

```
*****
* G4Track Information: Particle = Cs137, Track ID = 1, Parent ID = 0
*****
Step#    X(mm)    Y(mm)    Z(mm) KinE(MeV)  dE(MeV) StepLeng TrackLeng  NextVolume ProcName
0         0         0        -50      0         0         0         0 PhysVol_World initStep
1         0         0        -50      0         0         0         0 PhysVol_World RadioactiveDecay
*****
* G4Track Information: Particle = e-, Track ID = 4, Parent ID = 1
*****
Step#    X(mm)    Y(mm)    Z(mm) KinE(MeV)  dE(MeV) StepLeng TrackLeng  NextVolume ProcName
0         0         0        -50      0.169      0         0         0 PhysVol_World initStep
1    0.666    -1.76    -53.1    0.167 0.00745    3.65    3.65 PhysVol_World eIoni
2         33        -19    -111    0.145 0.019    72.8    76.4 PhysVol_World eIoni
3    33.5        -34    -135    0.137 0.00608    28.9    105 PhysVol_World eIoni
4    32.2    -42.4    -151    0.133 0.00249    18.8    124 PhysVol_World eIoni
5    32.3    -42.4    -152    0.132 9.08e-05    0.927    125 PhysVol_World eIoni
6    40.9    -44.1    -189    0.12 0.0101    41.5    167 PhysVol_World eIoni
7    34.5    -45.9    -214    0.11 0.00935    26.5    193 PhysVol_World eIoni
8    27.6    -40.6    -235    0.0584 0.00806    22.6    216 PhysVol_World eIoni
9    24.3    -41.8    -240    0.0398 0.00349    6.71    222 PhysVol_World eIoni
10   24.2        -38    -246    0.0303 0.00436    8.11    230 PhysVol_World eIoni
11   24.2    -37.9    -246    0.0196 4.64e-05    0.178    231 PhysVol_World eIoni
12   24.8    -35.4    -246    0.0163 0.00327    3.34    234 PhysVol_World eIoni
13   23.5    -35.9    -247    0.0123 0.00282    2.66    237 PhysVol_World eIoni
14   22.9    -36.8    -248    0.00875 0.00358    2.18    239 PhysVol_World eIoni
15   23.7    -36.4    -248    0.00485 0.00391    1.64    240 PhysVol_World eIoni
16   23.8    -36.7    -248      0 0.00485    0.725    241 PhysVol_World eIoni

===== 出力 一部省略 =====

*****
* G4Track Information: Particle = anti_nu_e, Track ID = 3, Parent ID = 1
*****
Step#    X(mm)    Y(mm)    Z(mm) KinE(MeV)  dE(MeV) StepLeng TrackLeng  NextVolume ProcName
0         0         0        -50      0.345      0         0         0 PhysVol_World initStep
1    446    -922    -1e+03    0.345      0 1.4e+03 1.4e+03 OutOfWorld Transportation
*****
* G4Track Information: Particle = Ba137[661.659], Track ID = 2, Parent ID = 1
*****
Step#    X(mm)    Y(mm)    Z(mm) KinE(MeV)  dE(MeV) StepLeng TrackLeng  NextVolume ProcName
0         0         0        -50    2.4e-06      0         0         0 PhysVol_World initStep
1 -0.000341 0.000963    -50      0 2.4e-06 0.00171 0.00171 PhysVol_World ionIoni
2 -0.000341 0.000963    -50      0      0 0.00171 0.00171 PhysVol_World RadioactiveDecay
*****
* G4Track Information: Particle = gamma, Track ID = 22, Parent ID = 2
*****
Step#    X(mm)    Y(mm)    Z(mm) KinE(MeV)  dE(MeV) StepLeng TrackLeng  NextVolume ProcName
0 -0.000341 0.000963    -50    0.662      0         0         0 PhysVol_World initStep
1 -1e+03    184    -527    0.662      0 1.12e+03 1.12e+03 OutOfWorld Transportation
*****
* G4Track Information: Particle = Ba137, Track ID = 21, Parent ID = 2
*****
Step#    X(mm)    Y(mm)    Z(mm) KinE(MeV)  dE(MeV) StepLeng TrackLeng  NextVolume ProcName
0 -0.000341 0.000963    -50 1.72e-06      0         0         0 PhysVol_World initStep
1 0.000948 0.000727    -50      0 1.72e-06 0.00145 0.00145 PhysVol_World ionIoni
2 0.000948 0.000727    -50      0      0 0.00145 0.00145 PhysVol_World RadioactiveDecay
*****
```



P08_PhysicsLists/Factory

Physics Listsのカスタマイズ: G4PhysListFactoryを使う

演習トピックス

[概要] [P08_PhysicsLists](#)プログラムを使い、物理過程のプロセスの設定方法を学ぶ

[目次]

1. [P08_PhysicsLists/Shielding](#)

- Radioactive Decayの扱い

2. [P08_PhysicsLists/Factory](#)

- Physics Listsのカスタマイズ: G4PhysListFactoryを使う

3. [P08_PhysicsLists/MyPhysList](#)

- 独自のPhysics Listsの作成

P08_PhysicsLists/Factoryプログラムの概要

■ 演習プログラムの目的

- 前演習では、ユーザが使用するReference Physics ListをRunManagerに直接渡すことでPhysics List設定を行った
 - ← 'FTFP_BERT' から 'Shielding' に変更し、放射性崩壊過程を扱えるようにした
- 本演習ではユーザが'G4PhysListFactory' クラスを使用し、既存のReference Physics Listの機能をカスタマイズしてPhysics Listとして設定する手法を学ぶ
 - ← この手法で、既存physics listsに物理過程を自由に追加、削除、置換できる
 - ← 演習課題では'FTFP_BERT' をカスタマイズして放射性崩壊過程を扱えるようにする
- 同じ手法でReference Physicsが採用している電磁相互作用モデルを変更する手法を学ぶ
 - ← ユーザが既存physics listsの電磁相互作用モデルを変更したい場合、これが推奨手法

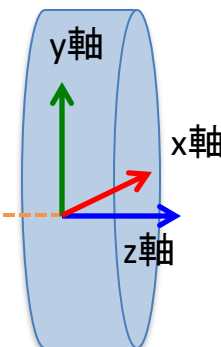
■ プログラムの構成

- mainプログラム: P08_PhysicsLists/Shieldingを変更
- ジオメトリ: P06_Geometryで使ったBGO_One検出器
- PhysicsList: G4PhysicsListFactoryを使って設定
- PrimaryGenerator: P08_PhysicsLists/Shieldingと同一

■ 組み込まれているジオメトリと粒子発生機能

Radioactive Source ^{137}Cs ✕
設定位置: $x = 0, y = 0, z = -5 \text{ cm}$

円筒(BGO):
座標系原点に配置



P08_PhysicsLists/Factory プログラムのビルド

課題:1 プログラム構成を理解する

- P08_PhysicsLists/ShieldingのApplication_Main.ccのみが次のように変更されている
- Shieldingを削除、G4PhysListFactoryクラスを用いてFTFP_BERTと放射性崩壊過程が設定される

課題:2 P08_PhysicsLists/Factoryのソースコードをビルドして実行する

1) P08_PhysicsLists/Factoryのディレクトリに移行する

```
$ cd ~/Geant4Tutorial_2025/P08_PhysicsLists/Factory
$ ls
source/  util/  TestBench/
```

← TestBenchのPrimaryGeneratorSetup.macには¹³⁷Csが設定されている

2) ソースをビルド

```
$ ./util/Build.sh
$ ls
bin/  build/  source/  util/  TestBench/
```

← Scriptを走らせるとアプリ実行環境が完成

← ソースのビルドでbin、buildが作られた

3) TestBench(作業ディレクトリ) に移行、プログラムを実行

```
$ cd TestBench
$ ../bin/Application_Main
```

P08_PhysicsLists/Factory プログラムを使う

課題:1 実行されたプログラムを使う

1) Qt画面のコマンド窓に以下を入力しアプリ実行

```
/control/execute GlobalSetup.mac
```

← または、Qtのファイルアイコンをクリック

2) Qtコマンド窓に以下のコマンドを入力

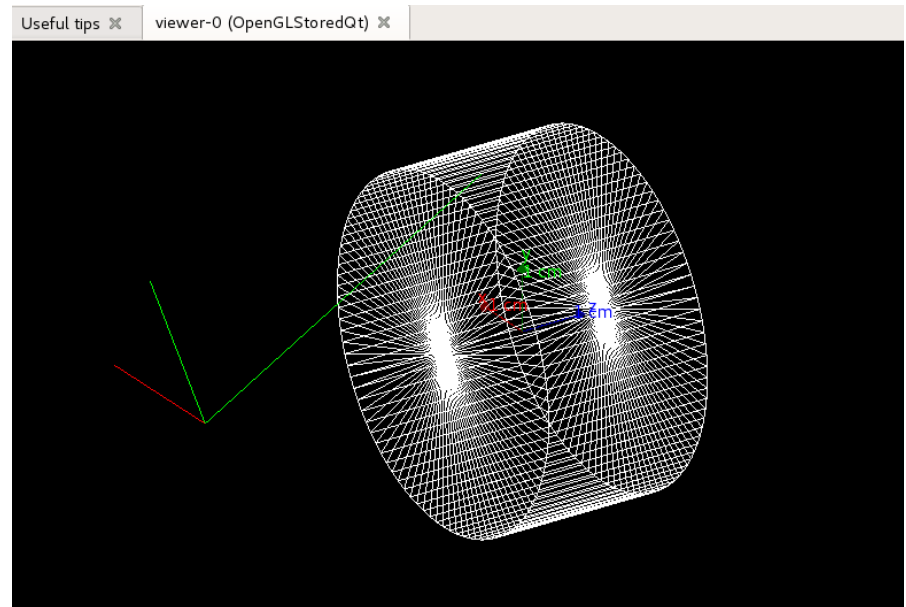
```
/run/beamOn 1
```

- 右図の様に、前演習のShielding'と同じく
放射性崩壊が組み込まれていることを確認

3) アプリを終了

```
exit
```

次の課題では、実行したプログラムの
[Application_Main.cc](#)のコードを読み、
実装手法を学ぶ



P08_PhysicsLists/Factoryのmainプログラムの構造

課題:2 G4PhysListFactoryを使った物理過程の追加方法をcolordiffを使い理解する

```
$ cd ../source
$ colordiff -y --width=160 ../util/G4Codes/P08_Application_Main.cc_Shielding Application_Main.cc
```

```
//+++++
// Geant4 Tutorial for Hep/Space/Medicine Users
//+++++
#include "Geometry.hh"
#include "UserActionInitialization.hh"

#include "G4RunManager.hh"
#include "G4UIManager.hh"
#include "G4VisExecutive.hh"
#include "G4UIExecutive.hh"
#include "Shielding.hh"
```

Shielding用のmain

変更
追加

```
//-----
int main( int argc, char** argv )
//-----
{
  Construct a run manager
  auto runManager = new G4RunManager();

  Set up mandatory user initialization: Geometry
  runManager->SetUserInitialization( new Geometry{} );

  Set up mandatory user initialization: Physics-List
  runManager->SetUserInitialization( new Shielding{} );

  Set up user initialization: User Actions
  runManager->SetUserInitialization( new UserActionInitialization{} );

  Initialize G4 kernel
  runManager->Initialize();

  Create visualization environment
  auto visManager = new G4VisExecutive{};
  visManager->Initialize();

  Start interactive session
  auto uiExec = new G4UIExecutive{ argc, argv };
  G4UIManager* uiManager = G4UIManager::GetUIpointer();
  uiManager->ApplyCommand( "/control/execute GlobalSetup.mac" );
  uiExec->SessionStart();

  Job termination
  delete uiExec;
  delete visManager;
  delete runManager;

  return 0;
}
```

1. G4PhysListFactoryを作る
2. そこから'FTFP_BERT'のPhysics Listを取得
3. 'new G4RadioactivePhysics()'を実行して放射性崩壊過程を生成
4. 過程をRegisterPhysics()でPhysics Listに追加
5. 変更したPhysics ListをRunManagerに渡す

```
//+++++
// Geant4 Tutorial for Hep/Space/Medicine Users
//+++++
#include "Geometry.hh"
#include "UserActionInitialization.hh"

#include "G4RunManager.hh"
#include "G4UIManager.hh"
#include "G4VisExecutive.hh"
#include "G4UIExecutive.hh"
#include "G4PhysListFactory.hh"
#include "G4VModularPhysicsList.hh"
#include "G4RadioactiveDecayPhysics.hh"
```

G4PhysListFactory用のmain

```
//-----
int main( int argc, char** argv )
//-----
{
  Construct a run manager
  auto runManager = new G4RunManager();

  Set up mandatory user initialization: Geometry
  runManager->SetUserInitialization( new Geometry{} );

  Set up mandatory user initialization: Physics-List
  G4PhysListFactory factory{};
  G4VModularPhysicsList* physicsList = factory.GetReferencePhysList( "FTFP_BERT" );
  physicsList->RegisterPhysics( new G4RadioactiveDecayPhysics{} );
  runManager->SetUserInitialization( physicsList );

  Set up user initialization: User Actions
  runManager->SetUserInitialization( new UserActionInitialization{} );

  Initialize G4 kernel
  runManager->Initialize();

  Create visualization environment
  auto visManager = new G4VisExecutive{};
  visManager->Initialize();

  Start interactive session
  auto uiExec = new G4UIExecutive{ argc, argv };
  G4UIManager* uiManager = G4UIManager::GetUIpointer();
  uiManager->ApplyCommand( "/control/execute GlobalSetup.mac" );
  uiExec->SessionStart();

  Job termination
  delete uiExec;
  delete visManager;
  delete runManager;

  return 0;
}
```

P08_PhysicsLists/Factory プログラムを使い電磁相互作用モデルを変更する

課題: 1 前課題のApplication_Main.ccで選択されている'FTFP_BERT'の電磁相互作用モデルを標準モデルからLivermoreモデル(低エネルギーモデル)に変更する方法を学ぶ

[注] 以下のステップの実行は自習用とし、現演習では実習しない

1) 現在のApplication_Main.ccを以下の様に変更する

```
//-----  
int main( int argc, char** argv )  
//-----  
{  
    // Construct a run manager  
    auto runManager = new G4RunManager{};  
  
    // Set up mandatory user initialization: Geometry  
    runManager->SetUserInitialization( new Geometry{} );  
  
    // Set up mandatory user initialization: Physics-List  
    G4PhysListFactory factory{};  
    G4VModularPhysicsList* physicsList = factory.GetReferencePhysList( "FTFP_BERT_LIV" );  
    physicsList->RegisterPhysics( new G4RadioactiveDecayPhysics{} );  
    runManager->SetUserInitialization( physicsList );  
}
```

← “FTFP_BERT”を“FTFP_BERT_LIV”に変更する

2) 変更後にアプリビルドし実行すると、ターミナル窓に以下の様なメッセージが表示される

```
*****  
Geant4 version Name: geant4-11-03 [MT]    (6-December-2024)  
    Copyright : Geant4 Collaboration  
    References : NIM A 506 (2003), 250-303  
                : IEEE-TNS 53 (2006), 270-278  
                : NIM A 835 (2016), 186-225  
                WWW : http://geant4.org/  
*****
```

```
G4PhysListFactory::GetReferencePhysList <FTFP_BERT_LIV>  EOption= 5  
<<< Geant4 Physics List simulation engine: FTFP_BERT
```

```
G4VModularPhysicsList::ReplacePhysics: G4EmStandard with type : 2 is replaced with G4EmLivermore
```

“FTFP_BERT_LIV”が設定されたという
メッセージ

P08_PhysicsLists/MyPhysList

独自のPhysics Listsの作成

演習トピックス

[概要] [P08_PhysicsLists](#)プログラムを使い、物理過程のプロセスの設定方法を学ぶ

[目次]

1. [P08_PhysicsLists/Shielding](#)
 - Radioactive Decayの扱い
2. [P08_PhysicsLists/Factory](#)
 - Physics Listsのカスタマイズ: G4PhysListFactoryを使う
3. [P08_PhysicsLists/MyPhysList](#)
 - 独自のPhysics Listsの作成



P08_PhysicsLists/MyPhysicsListプログラムの概要

■ 演習プログラムの目的

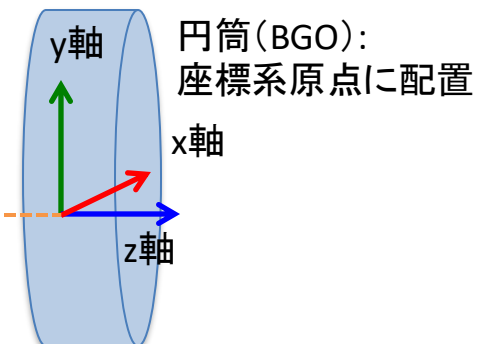
- この演習ではG4VModularPhysicsListクラスを使い、ユーザ独自のPhysics List作成法を学ぶ
 - ← 独自のPhysics Listを作る手法は簡単だが、そのPhysics Listを実用のシミュレーションに耐えるものにするには実データとの比較検証等による正当性のチェックが不可欠
 - ← 手法の応用例としては、ユーザが特定の物理過程のみを選択しシミュレーションしたい場合に使える
[例] 電磁相互作用のみを作動させ、その振る舞いを見してみる（以下の演習ではこの手法を示す）

■ プログラムの構成

- mainプログラム: P08_PhysicsLists/Shieldingを変更
- ジオメトリ: P06_Geometryで使ったBGO_One検出器
- PhysicsList: G4VModularPhysicsListを使いMyPhysicsList.hh/MyPhysicsList.ccを作成
 - ← 電磁相互作用過程 (standard EM processes) と粒子崩壊過程 (G4DecayPhysics, G4RadiativeDecay) のみが組み込まれている (Geant4提供の例題B3に基づく)
- PrimaryGenerator: P08_PhysicsLists/Shieldingと同一

■ 組み込まれているジオメトリと粒子発生機能

Radioactive Source ^{137}Cs ✕
設定位置: $x = 0, y = 0, z = -5 \text{ cm}$



P08_PhysicsLists/MyPhysicsList プログラムのビルド

課題:1 プログラム構成を理解する

- G4VModularPhysicsListクラスを継承してMyPhysicsListクラスを作成
- 作成したMyPhysicsListを前課題のApplication_Main.ccに登録

課題:2 P08_PhysicsLists/MyPhysicsListのソースコードをビルドして実行する

1) P08_PhysicsLists/MyPhysicsListのディレクトリに移行する

```
$ cd ~/Gean4Tutorial_2025/P08_PhysicsLists/MyPhysicsList
$ ls
source/  util/  TestBench/
```

← TestBenchのPrimaryGeneratorSetup.mac
には¹³⁷Csが設定されている

2) ソースをビルド

```
$ ./util/Build.sh
$ ls
bin/  build/  source/  util/  TestBench/
```

← Scriptを走らせるとアプリ実行環境が完成

← ソースのビルドでbin、buildが作られた

3) TestBench(作業ディレクトリ) に移行、プログラムを実行

```
$ cd TestBench
$ ../bin/Application_Main
```

P08_PhysicsLists/MyPhysList プログラムを使う

課題:1 実行されたプログラムを使う

1) Qt画面のコマンド窓に以下を入力しアプリ実行

```
/control/execute GlobalSetup.mac
```

← または、Qtのファイルアイコンをクリック

2) Qtコマンド窓に以下のコマンドを入力

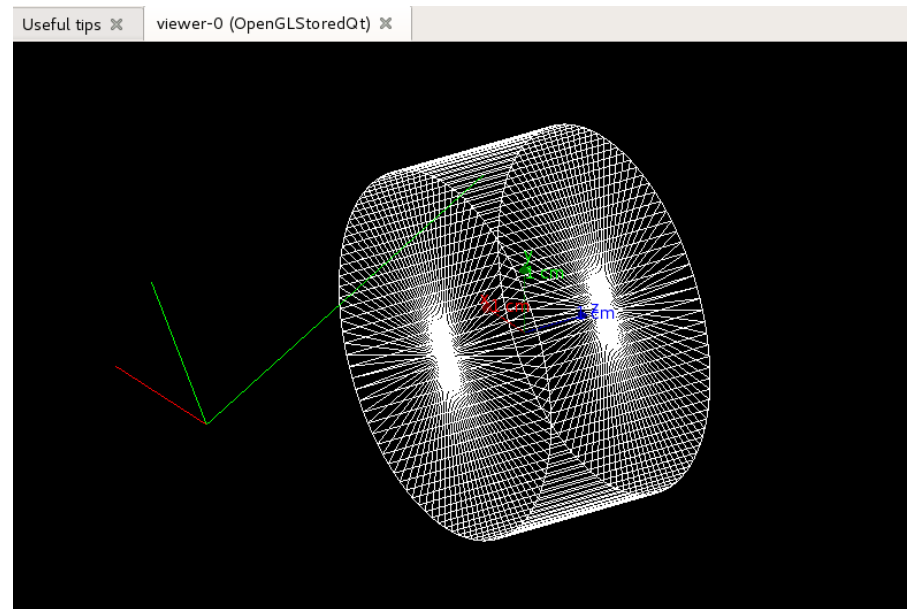
```
/run/beamOn 1
```

- 右図の様に、放射性崩壊が組み込まれていることを確認

3) アプリを終了

```
exit
```

次の課題では、実行したプログラムの
[Application_Main.cc](#)のコードを読み、
実装手法を学ぶ



P08_PhysicsLists/MyPhysicsListプログラムの構造の理解

課題:1 MyPhysicsList.hhとMyPhysicsList.ccがどのように実装されているかを理解する

```
$ cd source/include
$ less MyPhysicsList.hh
```

```
//+++++MyPhysicsList.hh+++++
// MyPhysicsList.hh
//+++++
#ifndef MyPhysicsList_h
#define MyPhysicsList_h 1

#include "G4VModularPhysicsList.hh"

//-----
class MyPhysicsList : public G4VModularPhysicsList
//-----
{
public:
    MyPhysicsList();
    ~MyPhysicsList();

public:
    void SetCuts();
};
#endif
```

ユーザ独自のPhysicsListクラス“MyPhysicsList”は
Geant4が用意している“G4VModularPhysicsList”クラス
を継承して定義する

“MyPhysicsList.cc” で使用したい物理過程を登録

```
$ cd ../src
$ less MyPhysicsList.cc
```

```
//+++++MyPhysicsList.cc+++++
// MyPhysicsList.cc
// [Note] Based on "G4 Basic Example: B3"
//+++++
#include "MyPhysicsList.hh"
#include "G4DecayPhysics.hh"
#include "G4RadioactiveDecayPhysics.hh"
#include "G4EmStandardPhysics.hh"

//-----
MyPhysicsList::MyPhysicsList()
: G4VModularPhysicsList{}
//-----
{
    // Default physics
    RegisterPhysics(new G4DecayPhysics{});

    // Radioactive decay
    RegisterPhysics(new G4RadioactiveDecayPhysics{});

    // EM physics
    RegisterPhysics(new G4EmStandardPhysics{});
}

//-----
MyPhysicsList::~MyPhysicsList()
//-----
{}

//-----
void MyPhysicsList::SetCuts()
//-----
{
    G4VUserPhysicsList::SetCuts();
}
```

使いたい物理過程
を登録する

P08_PhysicsLists/MyPhysicsListプログラムの構造の理解 - つづき

課題:2 Application_Main.ccにMyPhysicsListを登録する方法を理解

[注] 登録方法は以下のShielding用とMyPhysicsList用のApplication_Main.ccを比較を参照

```
$ cd ../source
$ colordiff -y --width=160 ../../Shielding/source/Application_Main.cc Application_Main.cc
```

Shielding用のmain	MyPhysicsList用のmain
<pre>// Geant4 Tutorial for Hep/Space/Medicine Users #include "Geometry.hh" #include "UserActionInitialization.hh" #include "G4RunManager.hh" #include "G4UIManager.hh" #include "G4VisExecutive.hh" #include "G4UIExecutive.hh" #include "Shielding.hh"</pre>	<pre>// Geant4 Tutorial for Hep/Space/Medicine Users #include "Geometry.hh" #include "UserActionInitialization.hh" #include "G4RunManager.hh" #include "G4UIManager.hh" #include "G4VisExecutive.hh" #include "G4UIExecutive.hh" #include "MyPhysicsList.hh"</pre>
<pre>//----- int main(int argc, char** argv) //----- { // Construct a run manager auto runManager = new G4RunManager (); // Set up mandatory user initialization: Geometry runManager->SetUserInitialization(new Geometry{}); // Set up mandatory user initialization: Physics-List runManager->SetUserInitialization(new Shielding{}); // Set up user initialization: User Actions runManager->SetUserInitialization(new UserActionInitialization{}); // Initialize G4 kernel runManager->Initialize(); // Create visualization environment auto visManager = new G4VisExecutive{}; visManager->Initialize(); // Start interactive session auto uiExec = new G4UIExecutive{ argc, argv }; G4UIManager* uiManager = G4UIManager::GetUIpointer(); uiManager->ApplyCommand("/control/execute GlobalSetup.mac"); uiExec->SessionStart(); // Job termination delete uiExec; delete visManager; delete runManager; return 0; }</pre>	<pre>//----- int main(int argc, char** argv) //----- { // Construct a run manager auto runManager = new G4RunManager(); // Set up mandatory user initialization: Geometry runManager->SetUserInitialization(new Geometry); // Set up mandatory user initialization: Physics-List runManager->SetUserInitialization(new MyPhysicsList{}); // Set up user initialization: User Actions runManager->SetUserInitialization(new UserActionInitialization{}); // Initialize G4 kernel runManager->Initialize(); // Create visualization environment auto visManager = new G4VisExecutive{}; visManager->Initialize(); // Start interactive session auto uiExec = new G4UIExecutive{ argc, argv }; G4UIManager* uiManager = G4UIManager::GetUIpointer(); uiManager->ApplyCommand("/control/execute GlobalSetup.mac"); uiExec->SessionStart(); // Job termination delete uiExec; delete visManager; delete runManager; return 0; }</pre>

演習 終了